SPRING BOOT

Spring — Bookstore Chapter 3: от пустого проекта до сдачи

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа; Extra выполнен дополнительно

Учебный комплект • 2026

> В этой инструкции сначала разобрана только работа Chapter 3. Готовый код в `projects/spring-boot/bookstore` является накопительным: поверх Chapter 3 в нём уже выполнены Chapter 4, Bootstrap Extra и отдельный MySQL Optional. Эти расширения отмечены ниже и не превращены в требования работы 03.

Что требуется

Обязательны разделы 1–4 исходного задания:

1. Подключить Spring Data JPA и H2, сделать `Book` entity-классом, создать `BookRepository` на основе `CrudRepository`, добавить несколько книг через `CommandLineRunner`, включить вывод SQL и H2 Console.
2. Создать `booklist.html`, вывести все книги в HTML-таблице и открыть страницу через GET-маршрут наподобие `/booklist`.
3. Реализовать создание книги через `addbook.html` и удаление книги по идентификатору с `@PathVariable`.
4. Реализовать редактирование текущей книги; ссылка Edit должна находиться после Delete.
5. После требуемых этапов отправить изменения в собственный репозиторий GitHub или репозиторий, выданный GitHub Classroom.

Для книги используются поля `id`, `title`, `author`, `isbn`, `publicationYear`. Поле `year` использовать не нужно: в условии предложено безопасное имя `publicationYear`.

Что не является обязательным критерием Chapter 3

- Bootstrap — это раздел 5, прямо названный **Extra**. В готовом комплекте он выполнен, но обязательные разделы 1–4 можно сдать без него.
- HTTP 404 для неизвестного идентификатора — полезное усиление надёжности, но исходное условие не задаёт такое поведение.
- `BookForm`, защита от подстановки `id` и схема Post/Redirect/Get — полезные улучшения готового решения, а не отдельные пункты исходного задания.
- Автоматические JPA- и MockMvc-тесты помогают проверить работу, но исходное условие не требует конкретных тестов.
- `Category`, связь one-to-many и выпадающий список относятся к Chapter 4.
- MySQL и XAMPP относятся к отдельной Optional-работе 09. Для Chapter 3 достаточно H2.

Что получится

Приложение запускает встроенный веб-сервер и in-memory базу H2. При старте `CommandLineRunner` сохраняет три книги. Браузер получает список, форму создания и форму редактирования через Spring MVC и Thymeleaf; сохранение и удаление выполняются через `BookRepository`.

Основной поток данных:

КОД · TEXT

```
GET /booklist
    → BookController
    → BookRepository.findAll()
    → H2
    → Model["books"]
    → templates/booklist.html
    → HTML в браузере

POST /save или POST /save/{id}
    → BookController
    → BookRepository.save(...)
    → H2
    → redirect:/booklist
```

Маршруты рекомендуемого варианта:

Метод и адрес	Назначение	Требование курса
`GET /booklist`	Показать все книги	Да
`GET /add`	Открыть `addbook.html` для новой книги	Да
`POST /save`	Создать книгу	Да, точный адрес может отличаться
`GET /edit/{id}`	Открыть текущую книгу для редактирования	Да
`POST /save/{id}`	Сохранить изменения книги	Да, точный адрес может отличаться
`GET /delete/{id}`	Удалить книгу	Да; нужен `@PathVariable`
`GET /h2-console`	Открыть H2 Console	Да

Подготовка на macOS

Нужна JDK 17 или новее. Проект компилируется с target Java 17. Отдельно устанавливать Maven не требуется: Spring Initializr создаёт Maven Wrapper `mvnw` и `mvnw.cmd`.

macOS/Linux:

КОД · BASH

```
java -version
git --version
```

Windows PowerShell:

КОД · POWERSHELL

```
java -version
git --version
```

Если команда `java` не найдена, сначала установите JDK. Если Java установлена, но IntelliJ IDEA её не видит, выберите JDK 17 или новее в Project Structure. Подробная подготовка macOS находится в `teaching/macos-setup.md`.

Создание с нуля

На Spring Initializr выберите:

Поле	Значение
Project	Maven
Language	Java
Spring Boot	3.5.16
Group	`com.example`
Artifact / Name	`bookstore`
Packaging	Jar
Java	17

Добавьте зависимости:

- Spring Web;
- Thymeleaf;
- Spring Data JPA;
- H2 Database.

Скачайте архив, распакуйте его и откройте папку `bookstore`, в которой непосредственно лежит `pom.xml`.

macOS/Linux:

КОД · BASH

```
cd bookstore
chmod +x mvnw
./mvnw test
```

Windows:

КОД · POWERSHELL

```
cd bookstore
.\mvnw.cmd test
```

Первый запуск скачивает Maven и зависимости, поэтому ему нужен интернет. После заполнения локального Maven-кэша обычная работа с H2 не требует внешнего сервера или API-ключа.

Структура файлов

КОД · TEXT

```
bookstore/
├── .mvn/wrapper/maven-wrapper.properties  # создаёт Spring Initializr
├── .gitignore
├── mvnw                                   # создаёт Spring Initializr
├── mvnw.cmd                               # создаёт Spring Initializr
├── pom.xml
└── src/
    └── main/
        ├── java/com/example/bookstore/
        │   ├── BookstoreApplication.java
        │   ├── Book.java
        │   ├── BookForm.java
        │   ├── BookRepository.java
        │   ├── BookstoreDataInitializer.java
        │   └── BookController.java
        └── resources/
            ├── application.properties
            └── templates/
                ├── booklist.html
                └── addbook.html
```

`BookForm` используется в рекомендуемом безопасном варианте из этой инструкции. Он не является самостоятельным требованием курса. Maven Wrapper генерируется целиком через Spring Initializr: перепечатывать его длинные служебные скрипты вручную не нужно.

Если выполняется Bootstrap Extra, дополнительно появляются:

КОД · TEXT

```
src/main/resources/static/css/
├── bootstrap.min.css
└── bootstrap-LICENSE.txt
```

Пошаговая реализация

Ниже приведены полные, а не сокращённые файлы самостоятельного Chapter 3. Их можно копировать целиком. Код намеренно не содержит `Category`, `CategoryRepository`, `categoryId` и MySQL.

6.1. pom.xml

`pom.xml` описывает координаты проекта, версию Java, зависимости и сборочный плагин. `spring-boot-starter-test` оставлен для стандартного стартового теста и необязательных дополнительных тестов; наличие этой зависимости само по себе не делает тесты критерием курса.

КОД · XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
```

```
        <version>3.5.16</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>bookstore</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>bookstore</name>
    <description>Учебное приложение Spring Boot Bookstore</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Назначение зависимостей:

- `spring-boot-starter-web` подключает Spring MVC и встроенный сервер;
- `spring-boot-starter-thymeleaf` позволяет возвращать HTML-шаблоны;
- `spring-boot-starter-data-jpa` подключает JPA, Hibernate и Spring Data;
- `h2` предоставляет встроенную базу данных во время запуска;
- `spring-boot-maven-plugin` собирает запускаемый JAR.

6.2. .gitignore

Сгенерированную папку `target`, локальные базы и файлы с секретами отправлять в GitHub не нужно.

КОД · GITIGNORE

```
target/
.env
.env.*
*.local.properties
*.mv.db
```

6.3. src/main/java/com/example/bookstore/BookstoreApplication.java

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookstoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(BookstoreApplication.class, args);
    }
}
```

`@SpringBootApplication` объединяет три задачи: помечает класс как конфигурацию, включает автоматическую настройку Spring Boot и запускает поиск компонентов в пакете `com.example.bookstore` и его подпакетах. Поэтому контроллер, репозитории и конфигурация должны находиться в этом пакете или ниже.

`main` — обычная точка входа Java. `SpringApplication.run` создаёт Spring-контекст, настраивает встроенный сервер, datasource, Hibernate, репозитории и MVC.

6.4. src/main/resources/application.properties

КОД · PROPERTIES

```
spring.application.name=bookstore
spring.datasource.url=jdbc:h2:mem:bookstore;MODE=MySQL;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Что делает каждая группа настроек:

- `jdbc:h2:mem:bookstore` создаёт базу в памяти; после завершения приложения данные исчезают;
- `MODE=MySQL` включает лишь режим совместимости синтаксиса внутри H2; внешний MySQL-сервер и MySQL Connector для Chapter 3 не нужны;
- `DB_CLOSE_DELAY=-1` не закрывает базу, пока работает JVM;
- `ddl-auto=create-drop` создаёт таблицы по entity-классам при старте и удаляет при остановке;
- `show-sql` и `format_sql` показывают сформированный Hibernate SQL в терминале;

- `spring.h2.console.*` включает веб-консоль по адресу `/h2-console`;
- пустое значение `spring.datasource.password=` означает пустой пароль, а не пропущенную настройку.

6.5. src/main/java/com/example/bookstore/Book.java

КОД · JAVA

```
package com.example.bookstore;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;
    private String isbn;
    private int publicationYear;

    public Book() {
    }

    public Book(Long id, String title, String author, String isbn, int publicationYear) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.publicationYear = publicationYear;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }
```

```
    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public int getPublicationYear() {
        return publicationYear;
    }

    public void setPublicationYear(int publicationYear) {
        this.publicationYear = publicationYear;
    }
}
```

Главные элементы entity:

- `@Entity` сообщает JPA, что экземпляры `Book` нужно сохранять в таблице;
- `@Id` отмечает первичный ключ;
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` поручает базе назначать новый идентификатор;
- `Long id`, а не `long`, позволяет новой несохранённой книге иметь `id == null`;
- открытый конструктор без аргументов нужен JPA для создания объекта при чтении строки из БД;
- getters и setters нужны JPA, Thymeleaf и привязке данных формы;
- конструктор со всеми полями удобен для стартовых примеров;
- `publicationYear` соответствует формулировке задания и не использует проблемное имя `year`.

6.6. src/main/java/com/example/bookstore/BookRepository.java

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.data.repository.CrudRepository;

public interface BookRepository extends CrudRepository<Book, Long> {
}
```

`CrudRepository<Book, Long>` означает: репозиторий работает с entity `Book`, а тип её идентификатора — `Long`. Реализацию интерфейса писать не нужно: Spring Data создаёт её во время запуска.

Используемые готовые методы:

Метод	Результат
`findAll()`	все книги
`findById(id)`	`Optional<Book>` с книгой либо без неё
`save(book)`	вставка новой или обновление существующей книги
`delete(book)`	удаление переданного объекта
`count()`	количество книг

6.7. src/main/java/com/example/bookstore/BookstoreDataInitializer.java

КОД · JAVA

```
package com.example.bookstore;

import java.util.List;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BookstoreDataInitializer {
    @Bean
    CommandLineRunner initialData(BookRepository bookRepository) {
        return arguments -> {
            if (bookRepository.count() > 0) {
                return;
            }

            Book cleanCode = new Book(null, "Clean Code", "Robert C. Martin", "9780132350884",
2008);
            Book effectiveJava = new Book(null, "Effective Java", "Joshua Bloch", "9780134685991",
2018);
            Book springInAction = new Book(null, "Spring in Action", "Craig Walls",
"9781617297571", 2022);

            bookRepository.saveAll(List.of(cleanCode, effectiveJava, springInAction));
        };
    }
}
```

`@Configuration` объявляет класс источником Spring-настроек. `@Bean` регистрирует возвращаемый объект в Spring-контексте. `CommandLineRunner` запускается один раз после готовности контекста и получает аргументы командной строки.

`BookRepository` попадает в метод через dependency injection: Spring видит требуемый тип и передаёт созданный репозиторий. Проверка `count()` не обязательна для одноразовой H2, но защищает от дублей, если позже используется постоянная база. У новых объектов `id` равен `null`, поэтому БД создаёт идентификаторы.

6.8. src/main/java/com/example/bookstore/BookForm.java

Этот класс — усиление безопасной привязки формы, уже используемое в готовом комплекте. В нём намеренно нет `id`: пользователь не может подставить идентификатор другой книги в POST `/save`. Исходное задание не требует отдельный form-класс, однако в рекомендуемом варианте ниже он является частью реализации.

КОД · JAVA

```
package com.example.bookstore;

public class BookForm {
    private String title;
    private String author;
    private String isbn;
    private int publicationYear;

    public String getTitle() {
        return title;
    }
```

```
    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public int getPublicationYear() {
        return publicationYear;
    }

    public void setPublicationYear(int publicationYear) {
        this.publicationYear = publicationYear;
    }
}
```

6.9. src/main/java/com/example/bookstore/BookController.java

Контроллер ниже закрывает обязательный CRUD и уже содержит два улучшения готового комплекта: отдельный `BookForm` и понятный HTTP 404 для неизвестного `id`. Они полезны, но при оценке исходной работы не должны выдаваться за дополнительные требования курса.

КОД · JAVA

```
package com.example.bookstore;

import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.server.ResponseStatusException;

@Controller
public class BookController {
    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @GetMapping("/booklist")
    public String bookList(Model model) {
        model.addAttribute("books", bookRepository.findAll());
        return "booklist";
    }

    @GetMapping("/add")
    public String addBookForm(Model model) {
        model.addAttribute("book", new Book());
```

```
        return "addbook";
    }

    @PostMapping("/save")
    public String addBook(@ModelAttribute BookForm form) {
        return saveBook(new Book(), form);
    }

    @PostMapping("/save/{id}")
    public String updateBook(@PathVariable Long id, @ModelAttribute BookForm form) {
        return saveBook(requiredBook(id), form);
    }

    private String saveBook(Book book, BookForm form) {
        book.setTitle(form.getTitle());
        book.setAuthor(form.getAuthor());
        book.setIsbn(form.getIsbn());
        book.setPublicationYear(form.getPublicationYear());
        bookRepository.save(book);
        return "redirect:/booklist";
    }

    @GetMapping("/edit/{id}")
    public String editBookForm(@PathVariable Long id, Model model) {
        model.addAttribute("book", requiredBook(id));
        return "addbook";
    }

    @GetMapping("/delete/{id}")
    public String deleteBook(@PathVariable Long id) {
        bookRepository.delete(requiredBook(id));
        return "redirect:/booklist";
    }

    private Book requiredBook(Long id) {
        return bookRepository.findById(id)
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Book not
found"));
    }
}
```

Как читать контроллер:

- `@Controller` регистрирует класс как MVC-контроллер; возвращаемая строка трактуется как имя шаблона или redirect;
- constructor injection передаёт единственный `BookRepository`; поле `final` показывает, что зависимость обязательна и не меняется;
- `@GetMapping` обрабатывает чтение страниц, а `@PostMapping` — отправку формы, изменяющую данные;
- `Model` — контейнер данных для одного HTML-ответа; `model.addAttribute("books", ...)` делает `${books}` доступным в Thymeleaf;
- `@ModelAttribute BookForm form` создаёт объект формы и заполняет его параметрами `title`, `author`, `isbn`, `publicationYear`;
- `@PathVariable Long id` берёт значение из участка URL `{id}`;
- один `saveBook` не дублирует копирование полей для create и edit;
- `save` вставляет книгу с `id == null`, но обновляет уже загруженную книгу с существующим `id`;
- `redirect:/booklist` реализует Post/Redirect/Get: после POST браузер выполняет новый GET, поэтому обновление страницы не повторяет сохранение;

- `requiredBook` и `ResponseStatusException` дают 404 вместо аварии с 500. Это enhancement, а не исходный критерий.

6.10. src/main/resources/templates/booklist.html

Это полная обязательная версия без Bootstrap Extra.

КОД · HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bookstore</title>
</head>
<body>
<main>
    <h1>Bookstore</h1>
    <p><a th:href="@{/add}">Add Book</a></p>
    <table>
        <thead>
        <tr>
            <th>ID</th>
            <th>Title</th>
            <th>Author</th>
            <th>ISBN</th>
            <th>Publication year</th>
            <th>Actions</th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="book : ${books}">
            <td th:text="${book.id}"></td>
            <td th:text="${book.title}"></td>
            <td th:text="${book.author}"></td>
            <td th:text="${book.isbn}"></td>
            <td th:text="${book.publicationYear}"></td>
            <td>
                <a th:href="@{/delete/{id}(id=${book.id})}">Delete</a>
                <a th:href="@{/edit/{id}(id=${book.id})}">Edit</a>
            </td>
        </tr>
        </tbody>
    </table>
</main>
</body>
</html>
```

Thymeleaf-конструкции:

- `xmlns:th` подключает пространство имён атрибутов Thymeleaf;
- `th:each="book : ${books}"` повторяет строку для каждой книги из `Model`;
- `th:text` безопасно помещает значение в текст ячейки;
- `${...}` обращается к данным модели и свойствам JavaBean через getters;
- `th:href="@{...}"` строит URL с учётом контекста приложения;
- `{id}` — место параметра пути, а `(id=${book.id})` подставляет идентификатор;
- Edit расположен после Delete, как требует формулировка задания.

6.11. src/main/resources/templates/addbook.html

Один полный шаблон используется и для создания, и для редактирования.

КОД · HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title th:text="${book.id == null ? 'Add Book' : 'Edit Book'}">Add Book</title>
</head>
<body>
<main>
    <h1 th:text="${book.id == null ? 'Add Book' : 'Edit Book'}">Add Book</h1>
    <form th:action="${book.id == null} ? @{/save} : @{/save/{id}(id=${book.id})}"
          th:object="${book}" method="post">
        <div>
            <label for="title">Title</label>
            <input id="title" type="text" th:field="*{title}" required>
        </div>
        <div>
            <label for="author">Author</label>
            <input id="author" type="text" th:field="*{author}" required>
        </div>
        <div>
            <label for="isbn">ISBN</label>
            <input id="isbn" type="text" th:field="*{isbn}" required>
        </div>
        <div>
            <label for="publicationYear">Publication year</label>
            <input id="publicationYear" type="number" th:field="*{publicationYear}" required>
        </div>
        <button type="submit">Save</button>
        <a th:href="@{/booklist}">Cancel</a>
    </form>
</main>
</body>
</html>
```

Как работает форма:

- `th:object="${book}"` выбирает объект, переданный контроллером;
- `th:field="*{title}"` одновременно задаёт `name`, текущее `value` и связь с полем выбранного объекта;
- `*{...}` — выражение относительно `th:object`, тогда как `${...}` — выражение от всей модели;
- тернарное выражение в `th:action` отправляет новую книгу в `/save`, а редактируемую — в `/save/{id}`;
- `method="post"` соответствует `@PostMapping`;
- HTML-атрибут `required` выполняет простую проверку в браузере, но не заменяет серверную валидацию;
- `label for` и совпадающий `input id` связывают подпись с полем.

Порядок реализации и промежуточные проверки

Работайте небольшими этапами, чтобы ошибка была связана с последним изменением.

Этап 1. Каркас и H2

1. Создайте проект через Initializr.
2. Замените `pom.xml` и `application.properties` полными версиями выше.
3. Оставьте или создайте `BookstoreApplication.java`.
4. Проверьте контекст:

macOS/Linux:

КОД · BASH

```
./mvnw test
```

Windows:

КОД · POWERSHELL

```
.\mvnw.cmd test
```

Этап 2. Entity, repository и начальные данные

1. Создайте `Book.java`.
2. Создайте `BookRepository.java`.
3. Создайте `BookstoreDataInitializer.java`.
4. Запустите приложение и убедитесь, что в терминале видны `create table book` и три `insert`.

Этап 3. Список

1. Создайте `BookController.java` сначала с `GET /booklist`.
2. Создайте `booklist.html`.
3. Откройте `http://localhost:8080/booklist` и проверьте три строки.

Этап 4. Create, Delete и Edit

1. Создайте `BookForm.java` и полный контроллер.
2. Создайте `addbook.html`.
3. Проверьте добавление, удаление и редактирование.
4. Убедитесь, что редактирование сохраняет прежний `id`, а не создаёт четвёртую копию.

Bootstrap Extra — необязательный раздел 5

Этот раздел выполняется только по желанию. Его отсутствие не означает, что обязательные разделы 1–4 не выполнены.

8.1. Локальный CSS

Скачайте полный `bootstrap.min.css` из дистрибутива Bootstrap и положите его в `src/main/resources/static/css`. Не подключайте CDN: условие Extra требует локальный файл.

В готовом комплекте полный неизменённый vendor-файл уже находится здесь:

КОД · TEXT

```
projects/spring-boot/bookstore/src/main/resources/static/css/bootstrap.min.css
```

Это Bootstrap 5.3.3, размер 232803 байта, SHA-256:

КОД · TEXT

```
3c8f27e6009ccfd710a905e6dcf12d0ee3c6f2ac7da05b0572d3e0d12e736fc8
```

Минифицированный vendor-файл не перепечатывается фрагментом: скопируйте его целиком из указанного пути или официального дистрибутива. Так в проект попадёт полный CSS, а не неполный учебный отрывок.

Полный файл лицензии:

КОД · TEXT

```
The MIT License (MIT)

Copyright (c) 2011-2024 The Bootstrap Authors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
THE SOFTWARE.
```

8.2. Полная Extra-версия booklist.html

Она отличается от обязательной версии локальным `<link>` и точными классами из задания: `table table-striped`, `btn btn-success`, `btn btn-danger`.

КОД · HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bookstore</title>
    <link type="text/css" rel="stylesheet" href="/css/bootstrap.min.css"
th:href="@{/css/bootstrap.min.css}">
</head>
<body>
<main class="container py-4">
    <h1 class="mb-4">Bookstore</h1>
    <p><a class="btn btn-success" th:href="@{/add}">Add Book</a></p>
    <table class="table table-striped">
        <thead>
        <tr>
            <th scope="col">ID</th>
            <th scope="col">Title</th>
            <th scope="col">Author</th>
            <th scope="col">ISBN</th>
            <th scope="col">Publication year</th>
```

```
            <th scope="col">Actions</th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="book : ${books}">
            <td th:text="${book.id}"></td>
            <td th:text="${book.title}"></td>
            <td th:text="${book.author}"></td>
            <td th:text="${book.isbn}"></td>
            <td th:text="${book.publicationYear}"></td>
            <td>
                <a class="btn btn-danger" th:href="@{/delete/{id}(id=${book.id})}">Delete</a>
                <a class="btn btn-outline-secondary"
th:href="@{/edit/{id}(id=${book.id})}">Edit</a>
            </td>
        </tr>
        </tbody>
    </table>
</main>
</body>
</html>
```

8.3. Полная Extra-версия addbook.html

Исходное Extra задаёт точные классы только для таблицы, Add Book и Delete. Остальные Bootstrap-классы ниже улучшают вид формы, но не являются отдельными критериями задания.

КОД · HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title th:text="${book.id == null ? 'Add Book' : 'Edit Book'}">Add Book</title>
    <link type="text/css" rel="stylesheet" href="/css/bootstrap.min.css"
th:href="@{/css/bootstrap.min.css}">
</head>
<body>
<main class="container py-4">
    <h1 th:text="${book.id == null ? 'Add Book' : 'Edit Book'}">Add Book</h1>
    <form th:action="${book.id == null} ? @{/save} : @{/save/{id}(id=${book.id})}"
          th:object="${book}" method="post" class="mt-4">
        <div class="mb-3">
            <label class="form-label" for="title">Title</label>
            <input class="form-control" id="title" type="text" th:field="*{title}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="author">Author</label>
            <input class="form-control" id="author" type="text" th:field="*{author}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="isbn">ISBN</label>
            <input class="form-control" id="isbn" type="text" th:field="*{isbn}" required>
        </div>
        <div class="mb-3">
            <label class="form-label" for="publicationYear">Publication year</label>
            <input class="form-control" id="publicationYear" type="number"
th:field="*{publicationYear}" required>
        </div>
        <button class="btn btn-success" type="submit">Save</button>
        <a class="btn btn-secondary" th:href="@{/booklist}">Cancel</a>
    </form>
</main>
</body>
```

```
</html>
```

Улучшения надёжности и тесты — уже есть, но не являются критериями источника

9.1. Понятный 404

Рекомендуемый `BookController` уже использует `requiredBook`. Для `/edit/999999`, `/delete/999999` и `POST /save/999999` он возвращает HTTP 404. Это лучше, чем `findById(id).get()`, которое для отсутствующей книги приводит к необработанному исключению, но исходная работа не требует отдельно демонстрировать 404.

9.2. Безопасный create и Post/Redirect/Get

`BookForm` не содержит `id`, поэтому POST `/save` всегда создаёт новый `Book`. При edit контроллер сначала загружает существующую entity по path variable, а затем копирует только разрешённые поля. Redirect после POST предотвращает повторную отправку формы при обновлении страницы. Это полезные свойства готового решения, не дополнительные критерии курса.

9.3. Автоматические тесты готового комплекта

Накопительный проект содержит:

- `BookRepositoryTest` для сохранения полей книги;
- `BookControllerTest` для списка, форм, create/edit/delete, 404 и локального CSS;
- `BookstoreDataInitializerTest` для защиты от повторного наполнения;
- `BookstoreApplicationTests` для контекста и конфигурации;
- `CategoryRepositoryTest` только для следующего Chapter 4.

Создавать эти тесты для выполнения исходных разделов 1–4 не обязательно. Если они уже находятся в выданном проекте, они должны проходить:

macOS/Linux:

КОД · BASH

```
./mvnw test
```

Windows:

КОД · POWERSHELL

```
.\mvnw.cmd test
```

Почему готовый проект отличается от чистого Chapter 3

Путь `projects/spring-boot/bookstore` содержит итог нескольких последовательных работ. Полные блоки выше — самостоятельная проекция Chapter 3, которая компилируется без файлов следующих глав.

Файл готового проекта	Совпадение с блоком выше	Накопительное отличие
`.gitignore`	Точное	Нет
`BookstoreApplication.java`	Точное	Нет
`BookForm.java`	Точное	Нет

Файл готового проекта	Совпадение с блоком выше	Накопительное отличие
`BookRepository.java`	Точное	Нет
`application.properties`	Точное	Нет
`pom.xml`	Проекция Chapter 3	В готовом проекте дополнительно есть MySQL Connector/J для Optional 09
`Book.java`	Проекция Chapter 3	В готовом проекте есть `@ManyToOne Category` из Chapter 4
`BookstoreDataInitializer.java`	Проекция Chapter 3	В готовом проекте сначала создаются категории и связи Chapter 4
`BookController.java`	Проекция Chapter 3	В готовом проекте внедрён `CategoryRepository` и принимается `categoryId`
`booklist.html`	Проекция Chapter 3/Extra	В готовом проекте добавлен столбец Category
`addbook.html`	Проекция Chapter 3/Extra	В готовом проекте добавлен dropdown Category
`bootstrap.min.css` и лицензия	Точные vendor-файлы	Нет

Для работы 03 не создавайте `Category.java`, `CategoryRepository.java` и `application-mysql.properties`. Не добавляйте `mysql-connector-j`. Они понадобятся в отдельных последующих работах.

Запуск и тесты

macOS/Linux

КОД · BASH

```
./mvnw test
./mvnw package
./mvnw spring-boot:run
```

Windows

КОД · POWERSHELL

```
.\mvnw.cmd test
.\mvnw.cmd package
.\mvnw.cmd spring-boot:run
```

Ручная проверка

Откройте `http://localhost:8080/booklist`.

H2 Console доступна по адресу `http://localhost:8080/h2-console`:

Поле	Значение
Driver Class	`org.h2.Driver`
JDBC URL	`jdbc:h2:mem:bookstore`
User Name	`sa`
Password	оставить пустым

В консоли выполните:

КОД · SQL

```sql
SELECT * FROM BOOK;
```

Остановить приложение можно сочетанием `Control+C`.

Обязательная ручная проверка Chapter 3

1. На `/booklist` видны три начальные книги и столбцы ID, Title, Author, ISBN, Publication year, Actions.
2. Через Add Book создаётся новая книга; после сохранения браузер возвращается к списку.
3. Delete удаляет выбранную книгу.
4. Edit открывает текущие значения; изменение сохраняется у той же книги.
5. Ссылка Edit стоит после Delete.
6. H2 Console открывается, а `SELECT * FROM BOOK;` показывает те же записи.
7. В терминале виден форматированный SQL Hibernate.

Дополнительная проверка enhancements

1. `/edit/999999` и `/delete/999999` возвращают 404.
2. Обновление страницы после сохранения не повторяет POST.
3. Подставленный параметр `id` в POST `/save` не перезаписывает существующую книгу.

Эти проверки полезны для качества, но не добавляйте их в список обязательных требований исходного задания.

Проверка Bootstrap Extra

1. `/css/bootstrap.min.css` открывается локально и не ведёт на CDN.
2. Таблица имеет `class="table table-striped"`.
3. Add Book имеет `class="btn btn-success"`.
4. Delete имеет `class="btn btn-danger"`.

Сдача через GitHub Classroom или собственный GitHub

Важная граница текущего комплекта

`fullstack-homework-kit` в этом рабочем пространстве не является самостоятельным Git-репозиторием для сдачи. Он лежит внутри защищённого index-only репозитория, который не должен отслеживать payload учебных проектов. Поэтому не пытайтесь отправить весь комплект как работу студента и не добавляйте его исходники принудительно в корневой индекс.

Для сдачи используйте отдельный репозиторий: выданный GitHub Classroom либо собственный репозиторий Bookstore. Переносите только папку проекта, а не весь `fullstack-homework-kit`.

Вариант A. GitHub Classroom

1. Откройте ссылку преподавателя и примите задание.
2. Скопируйте SSH-адрес созданного репозитория.
3. Клонируйте его в отдельную папку:

КОД · BASH

```
git clone git@github.com:ORGANIZATION/ASSIGNMENT-REPOSITORY.git
cd ASSIGNMENT-REPOSITORY
```

4. Если Classroom дал starter-код, вносите изменения в него. Если репозиторий пустой, поместите в него Maven Wrapper и только файлы Chapter 3 из раздела «Структура файлов», чтобы `pom.xml` оказался в корне репозитория. Не копируйте вслепую накопительные `Category*` и `application-mysql.properties` из готового комплекта, если преподаватель ждёт изолированную работу 03.
5. Выполните тест, сборку и ручную проверку.
6. Проверьте состав коммита и отправьте изменения. Команда `git add ... src` безопасна только после того, как вы убедились, что в `src` нет файлов последующих работ:

КОД · BASH

```
git status
git add .gitignore .mvn mvnw mvnw.cmd pom.xml src
git status
git commit -m "Реализована работа 03: Spring Bookstore"
git push
```

7. Откройте репозиторий на GitHub и убедитесь, что последний commit виден.

Вариант B. Собственный репозиторий

1. Создайте на GitHub отдельный пустой репозиторий Bookstore без секретов.
2. Клонируйте его по SSH и добавьте Maven Wrapper и чистые Chapter 3-файлы из этой инструкции в корень:

КОД · BASH

```
git clone git@github.com:USERNAME/BOOKSTORE-REPOSITORY.git
cd BOOKSTORE-REPOSITORY
```

3. Выполните те же команды `git add`, `git commit` и `git push`, что приведены выше.

4. Если преподавателю нужна ссылка, отправьте URL страницы репозитория, а не SSH-ключ и не локальный путь.

Что нельзя хранить в репозитории

- приватный или публичный SSH-ключ;
- personal access token, пароль GitHub или файл учётных данных;
- `.env`, локальные properties с паролями;
- папку `target` и собранные JAR-файлы;
- локальные файлы H2 `*.mv.db`;
- настройки аутентификации Git/WSL.

Git использует ключ из пользовательского окружения или менеджера учётных данных. Ключ не копируется в проект, не показывается в терминальном отчёте и не коммитится.

Перед сдачей обязательно проверьте:

КОД · BASH

```
git status
git log -1 --oneline
git ls-files
```

В `git ls-files` должны быть исходники, шаблоны, `pom.xml` и Maven Wrapper, но не `target`, `.env`, ключи или пароли.

Типичные ошибки

- **Приложение не находит классы.** Все Java-файлы должны начинаться с `package com.example.bookstore;` и лежать в соответствующей папке.
- **Таблица не создаётся.** Проверьте `@Entity`, `@Id`, H2/JPA dependencies и отсутствие поля `year`.
- **JPA не может создать Book.** Нужен открытый конструктор без аргументов.
- **Новая книга не получает id.** Проверьте `@GeneratedValue` и передавайте `null` для id нового объекта.
- **Шаблон не найден.** HTML должен лежать в `src/main/resources/templates`, а метод возвращать имя без `.html`.
- **Список пустой.** Проверьте, что `CommandLineRunner` зарегистрирован через `@Bean` и вызывает `saveAll`.
- **Поля формы пусты при edit.** Нужны `th:object="${book}"`, getters и `th:field`.
- **Edit создаёт новую строку.** Форма edit должна отправлять существующий `{id}`, а контроллер — сначала загрузить существующую entity.
- **POST повторяется после обновления.** Возвращайте `redirect:/booklist`, а не шаблон напрямую.
- **H2 Console не подключается.** Используйте именно JDBC URL из `application.properties`, user `sa`, пустой password.
- **Bootstrap не загружается.** Extra-файл должен лежать в `resources/static/css`, а URL — `/css/bootstrap.min.css`.
- **Проект случайно зависит от MySQL.** Для чистого Chapter 3 удалите Connector/J и `application-mysql.properties`; обычный запуск должен работать только на H2.

Вопросы и ответы

Почему Spring создаёт `BookRepository`, хотя класса реализации нет? Spring Data видит интерфейс, его entity и тип id, после чего создаёт proxy с готовыми CRUD-операциями.

Чем entity отличается от `Model`? Entity описывает сохраняемую запись базы данных, а MVC `Model` временно переносит данные контроллера в один HTML-шаблон.

Почему для новой книги передаётся `id == null`? Тогда Hibernate понимает, что нужна вставка, а `@GeneratedValue` позволяет базе назначить ключ.

Зачем нужен конструктор без аргументов? JPA создаёт объект через reflection, а затем заполняет его поля; без доступного no-args конструктора загрузка entity может завершиться ошибкой.

Почему после POST возвращается redirect? Браузер получает новый GET `/booklist`, поэтому обычное обновление страницы не отправляет форму повторно. Это полезное улучшение PRG, а не отдельный критерий источника.

Обязательны ли 404 и автоматические тесты? Нет. Они уже реализованы в готовом комплекте как enhancements и повышают качество, но исходные разделы 1–4 не задают их как критерии.

Почему в готовом Bookstore есть Category и MySQL? Это накопительный проект: Category относится к Chapter 4, а MySQL-профиль — к Optional 09. Чистое решение работы 03 от них не зависит.

Защита

Будьте готовы без чтения кода по строкам объяснить:

1. `Book` описывает данные, `BookRepository` выполняет CRUD, `BookController` связывает HTTP с репозиторием, `Model` передаёт данные в Thymeleaf.
2. `@Entity`, `@Id` и `@GeneratedValue` связывают Java-объект с таблицей и первичным ключом.
3. Spring сам создаёт реализацию `CrudRepository` и передаёт её контроллеру через constructor injection.
4. GET получает страницу или форму, POST изменяет данные.
5. `@PathVariable` берёт id из URL; `@ModelAttribute` собирает данные формы.
6. `CommandLineRunner` заполняет БД после старта приложения.
7. `th:each` создаёт строки таблицы, `th:text` выводит значения, `th:field` связывает input с объектом, `th:action` выбирает адрес отправки.
8. H2 удобна для учебной работы, потому что запускается вместе с приложением и не требует внешнего сервера.

Если преподаватель спрашивает о 404, `BookForm`, PRG или тестах, объясните их как уже реализованные улучшения качества. Если спрашивает о Category или MySQL, уточните, что это следующие отдельные работы, уже накопительно присутствующие в готовом комплекте.

Чек-лист сдачи

Обязательные критерии исходного Chapter 3

- ☐ Подключены Web, Thymeleaf, Data JPA и H2; target Java 17.
- ☐ `Book` является entity и содержит `publicationYear`, а не `year`.
- ☐ `BookRepository` расширяет `CrudRepository<Book, Long>`.
- ☐ `CommandLineRunner` создаёт несколько книг.
- ☐ SQL logging и H2 Console включены.
- ☐ `/booklist` показывает все поля книг в HTML-таблице.
- ☐ `addbook.html` и создание книги работают.
- ☐ Удаление использует `@PathVariable`.
- ☐ Edit открывает и сохраняет текущую книгу; ссылка стоит после Delete.

Необязательные улучшения готового решения

- ☐ Неизвестные id дают понятный HTTP 404.
- ☐ `BookForm` не позволяет подставить id при создании.
- ☐ После POST выполняется redirect.
- ☐ Автоматические тесты проходят.

Bootstrap Extra, только если выбран

- ☐ Полный Bootstrap хранится локально вместе с лицензией.
- ☐ Таблица использует `table table-striped`.
- ☐ Add Book использует `btn btn-success`.
- ☐ Delete использует `btn btn-danger`.

Перед отправкой

- ☐ `./mvnw test` и `./mvnw package` или Windows-аналоги завершаются успешно.
- ☐ Обязательный ручной сценарий пройден на H2.
- ☐ Проект находится в отдельном GitHub Classroom/собственном репозитории, а не в index-only корне комплекта.
- ☐ В commit нет `target`, `.env`, ключей, токенов, паролей и локальных файлов БД.
- ☐ Последний commit отправлен на GitHub и виден преподавателю.